Self-Adaptive Systems-on-Chips: Organization, Requirements, and Modeling Framework

Anderson Roberto Pinheiro Domingues, Fernando Gehm Moraes
School of Technology, Pontifical Catholic University of Rio Grande do Sul (PUCRS) – Porto Alegre, Brazil
anderson.domingues@pucrs.br, fernando.moraes@pucrs.br

***Abstract*—Sensor-rich embedded applications run under hard timing, power, and dependability constraints while responding to variations in workload and resource availability. Self-adaptation organizes sensor, decision-making, and actuation across hardware and software layers. This paper presents a modeling framework for handling self-adaptiveness in sensor-rich embedded applications, adopting concepts from prior literature on self-adaptive systems-on-chips (SoCs). A case study of a representative application, i.e., a deadline-aware task migration mechanism running on a manycore platform, indicates that the proposed framework encompasses self-adaptiveness across multiple system layers.**

***Index Terms*—Self-* Systems, Resource Management, Cyber-Physical Systems, Systems-on-Chips, Task Reallocation.**

I. Introduction

Sensor-rich embedded systems combine computation, communication, and physical interaction to support applications in domains such as robotics, cyber-physical systems, and the Internet of Things (IoT) [1,2]. Such systems must interpret streams of data from multiple sensors, identify abnormal situations or optimization opportunities, and reconfigure their operation at runtime while respecting design constraints.

Self-adaptation provides a way to structure this behavior. A self-adaptive system monitors its own operation, detects deviations from expected behavior, and autonomously takes corrective actions [3]. Implementing self-adaptation requires infrastructure beyond decision algorithms, including hardware sensing and actuation mechanisms, operating-system support, communication primitives, and reusable software abstractions [4]. While dealing with massive sensor data in embedded applications, the need for low-overhead solutions arises.

Multiple SoCs implement self-adaptive mechanisms, often aiming at application-level sensing/actuation and system resource management. This paper surveys self-adaptive SoCs, namely ASoC [5], CARUSO [6], HaMSoC [7], CPSoC [8], DodOrg [9], MEMPHIS [10], and ORCA [11], for design requirements that capture the infrastructure needed by self-adaptive systems. The **main contribution** of this paper is a modeling framework for self-adaptive systems built on top of such requirements.

The rest of this paper is organized as follows. Section II discusses surveyed studies, while Section III discusses the requirements for self-adaptiveness. Based on these requirements, this paper develops a modeling framework for structuring self-adaptiveness across hardware/software layers, which is introduced in Section IV. The paper presents two applications of the framework, discussing a deadline-aware task migration mechanism in a manycore platform (Section V). Finally, Section VI concludes the paper while presenting further research directions and technical improvements.

II. Background and Related Work

Self-adaptation encompasses sensing, actuation, and decision logic [12], often resembling Observe-Decide-Act (ODA) loops or control systems, and permeates the hardware and software components of a system. The literature presents a comprehensive set of studies on self-* properties [13] of self-adaptive systems, to which SoCs might adhere by either employing automated management of system resources or providing a testbed for implementing self-adaptive applications.

The Autonomic System-on-Chip (ASoC) [5] introduces a framework for developing SoC platforms with self-adaptive features. Two layers separate concerns: the *functional layer* implements application requirements, while the *autonomic layer* groups hardware dedicated to self-adaptation, i.e., monitors, actuators, and evaluators. Subsequent studies extended ASoC, including error-detection techniques as monitors [14], fine-tuning actuator behavior [15], and automating load balancing via hardware monitors and DVFS-based actuators [16].

The Connective Autonomic Real-time Ultra-low-power SoC (CARUSO) [6] targets SoC design with low energy consumption, connectivity, and real-time support. Most autonomic mechanisms rely on software and middleware, orchestrated by *helper threads* running on a dedicated multi-threaded processor. These threads are responsible for monitoring and decision-making, using fixed-time guaranteed-percentage scheduling to avoid interfering with application tasks.

The Hierarchical Agent Monitored SoC (HaMSoC) [7] comprises *a platform agent* for global optimization, *a cluster agent* for configuring clusters of processing element (PE), and *a cell agent* for monitoring individual PEs. A reported application uses hierarchical power management, in which cell agents communicate power requirements to cluster agents, which apply dynamic voltage-frequency scaling (DVFS) to reduce power without degrading quality-of-service (QoS).

The Cyber-Physical SoC (CPSoC) [8] employs cross-layer sensing and actuation to implement the ODA loop. It has two networks-on-chip (NoC): the *cNoC* for application packets and the *sNoC* for control. CPSoC offers an extensive set of physical sensors (e.g., circuit delay, aging, and temperature) and logical sensors (e.g., workload, execution time, and context-switch counters). Actuators include task migration, adaptive routing, DVFS, adaptive body biasing, and clock/power gating.

DodOrg [9] follows the organic computing paradigm, modeling components as the organs of the human body. Three tiers are used: *brain* (global coordination), *organ* (subsystem execution), and *cell*, i.e., individual PEs. Cells communicate through the artNoC [17]. Thermal management relies on dedicated clock domains that adapt frequency to power budget.

MEMPHIS [10] is a family of manycore systems built on top of the Hermes NoC. Its 2D mesh-based topology connects PEs, each comprising a network interface with DMA capabilities (DMNI), local scratchpad RAM, and a RISC-V RV32IMAC processor. This manycore has distributed management, with management tasks running at the user level. Each PE runs a multitasking operating system. A *task repository* stores application images for dynamic loading and task migration.

Finally, the ORCA manycore [11] is built on top of the Hermes NoC and the HF-RiscV processor [18]. It includes a software library for accessing hardware-level monitors via memory-mapped registers, enabling custom decision-making algorithms. Custom hardware can bind to registers at design time, extending the monitoring capabilities. This paper uses ORCA to validate the framework proposed herein.

III. Requirements for Self-Adaptiveness In SoCs

When comparing the aforementioned SoC platforms, we derived nine requirements for a general-purpose development platform targeting self-adaptive, sensor-rich embedded applications. Table I summarizes these requirements and how the literature addresses them, where *RQ1–RQ3* relate to hardware design, *RQ4–RQ5* regard kernel capabilities, *RQ6–RQ8* relate to software, and *RQ9* links to system-wide architecture.

RQ1: Hardware Sensing – allows enabling/disabling hardware sensors (runtime or design-time), as well as monitoring hardware (runtime only). Examples of hardware sensors include temperature, cycle counters, bus bandwidth consumption, and power monitors. The set of available sensors is platform-dependent. More sensors allow greater observability of the hardware at the cost of increased hardware complexity.

RQ2: Hardware Actuation – includes DVFS, power and clock gating, swappable routing algorithms, and NoC reconfiguration. The available actuators are also platform-dependent. Actuation tends to be limited when compared to sensing, as the variety of sensors is larger in surveyed works, and some actuation often requires intervention from components outside the system, e.g., active heat dissipation.

RQ3: Hardware Heterogeneity – includes support for mixed micro-architectures as well as an interface for communicating with external systems, e.g., including new peripherals. Some systems support multiple clock domains, e.g., for asymmetrical processors where tasks migrate to allow dark silicon.

RQ4: Real-Time (RT) Scheduling – allows auxiliary monitoring threads to run without degrading system QoS. For instance, task deadline violations can be a metric for further actions, such as frequency scaling in firm RT systems. Deploying RT kernels often relies on specialized hardware such as configurable counters and an interrupt subsystem.

RQ5: Inter-Process Communication (IPC) – enables kernel and application tasks to implement distributed self-adaptive control. An example is the hierarchical management, with management tasks running in user space. Unmanaged shared-memory spaces are often used in bare-metal environments.

RQ6: Software Sensing – includes software components for inspecting application and kernel software data, as well as providing libraries for hardware monitoring. Processor cores reporting utilization and overall workload are examples of monitorable metrics available only at the software level.

RQ7: Sensor Fusing – allows filtering data from sensors while combining multiple sensors to create more complex monitoring settings. Applying a Kalman filter to remove noise from raw sensor data is an example of filtering.

RQ8: Software Actuation – provides infrastructure for task-level actuation, including task reallocation and real-time parameter adjustment. It also acts as a layer for controlling hardware actuators, e.g., via memory-mapped registers.

RQ9: Architectural Flexibility – employs centralized, distributed, hierarchical, and mixed software architectures for resource management. Surveyed studies employ multi-layer decision-making.

TABLE I
Requirements As Addressed by Self-Adaptive SoCs.

Requirement / Platforms	ASoC [5]	CARUSO [6]	CPSoC [8]	DodOrg [9]	HaMSoC [7]	MEMPHIS [10]	ORCA [11]
RQ1: Hardware Sensing	●	●	●	●	●	●	●
RQ2: Hardware Actuation	●	●	●	●	●	●	S
RQ3: Hardware Heterogeneity	?	●	S	S	?	●	no
RQ4: Real-Time Scheduling	?	●	●	?	?	●	●
RQ5: Inter-Process Comm.	?	●	●	?	?	●	●
RQ6: Software Sensing	●	?	●	?	●	?	●
RQ7: Sensor Fusing	?	?	●	?	●	?	●
RQ8: Software Actuation	●	●	●	●	no	●	●
RQ9: Architectural Flexibility	●	?	●	no	●	no	?

● implemented; (S) partially implemented; (?) unknown; (no) unsupported

IV. Toward a Modeling Framework

This paper proposes a framework for organizing components in self-adaptive systems, aligned with the SoC domain. The framework categorizes components into layers, dimensions, and roles. Such a categorization enforces separation of concerns between components, contributing to the scalability and maintenance of self-adaptive architectures. Figure 1 illustrates the overall organization of the framework.

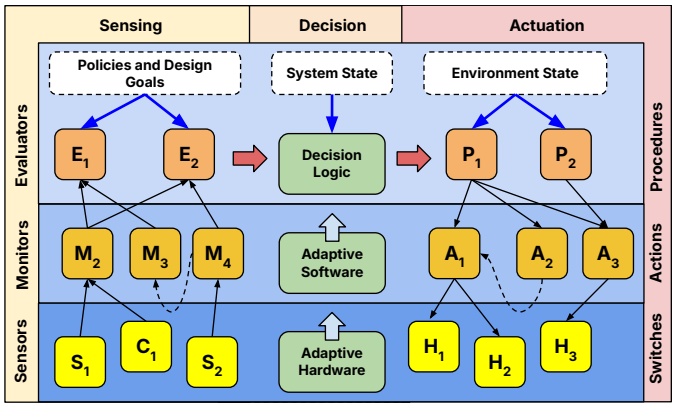

Fig. 1. Overview of the proposed framework. Black arrows denote data flow between components, and blue arrows denote design dependencies.

A. The hardware, software, and decision layers

Self-adaptive systems require infrastructure to support adaptation, which the proposed framework distributes across software, hardware, and decision layers.

The *hardware layer* encompasses device-level support for self-adaptation, including physical sensors (S_i) and counters (C_i), either as part of the SoC or as peripherals, and situational hardware (H_i), such as on-chip voltage regulators needed by DVFS techniques, e.g., [19].

The *software layer* abstracts hardware access through device drivers and the operating system, providing an interface for programming decision logic. The two building blocks of the software layer include monitors (M_i) and actions (A_i). Monitors comprise software for inspecting the system state, roughly representing the observation activity in ODA loops. Their counterparts, actions, activate underlying hardware and kernel directives, e.g., accessing the scheduler.

The *decision layer* implements the "intelligence" of self-adaptation through evaluators, integrating (E_i) and procedures (P_i). Evaluators observe monitors, detecting abnormal system operation or optimization opportunities. Complementary, procedures implement countermeasures and optimization algorithms, regulating the state of the system through actions. While actions are reusable atomic routines, procedures are high-level entities that may use one or more actions to apply decision logic.

B. The Sensing, Decision, and Actuation dimensions

The *sensing dimension* defines the system's observability, where counters, sensors, and monitors collect data, and evaluators check whether the data indicate an abnormal operational scenario. Evaluators use design goals, e.g., power and frequency constraints, along with user-defined policies, e.g., "the system must meet application deadlines in 97% of the time". Once evaluators determine abnormal operation or optimization opportunities, decision logic takes action.

The *decision dimension* determines the sequence of procedures required to move the system from its current state to a target state while accounting for possible state changes during planning. Decision logic may range from simple if-else control systems to robust machine learning models. From the decision logic viewpoint, the environment can be perceived only through evaluators, and procedures are the only tools decision logic has to influence it. Most of the time, the self-adaptive system will affect only parts of itself, although the proposed framework can be used to model interactions with external systems, as seen in Section V-B.

Finally, the *actuation dimension* selects procedures to create a plan. Plans may consider the resource-aware, potentially parallel execution of procedures. To do so, procedures may not share non-atomic actions, risking race conditions when they operate in user space. Actions, on the other hand, may run in kernel mode. In bare-metal software, procedures and actions are merely a matter of semantics.

V. Evaluation

This paper uses the ORCA manycore platform to demonstrate how the proposed framework models self-adaptive systems for a task reallocation subsystem.

A. The ORCA Manycore

ORCA [11] is a manycore platform targeting sensor-rich embedded applications, addressing 6 of the 9 requirements discussed earlier (see Table I). ORCA uses a 2D-mesh NoC for intra-chip communication. Parameters such as the number of PEs, memory (RAM and buffers) width and depth, and routing algorithms can be configured at design time. There are two kinds of *tiles* in the platform. First, the processing tiles comprise RAM and ROM, a single HFRiscV processor, and a network interface that attaches to the NoC router. There is also a networking tile comprising a network adapter that connects the NoC router to a UART, which connects the design to an external Ethernet MAC.

Applications run on top of the HellfireOS kernel in ORCA. Besides kernel directives, the platform also provides two key libraries: (i) one for assessing sensors, counters, and specialized hardware through the hardware abstraction layer (HAL), and (ii) a library for enabling publish-subscribe communication as IPC. The platform also has a built-in simulator, URSA, for cycle-accurate simulation. The platform has been validated in Questa software and prototyped to an ARTIX-7 FPGA using the Nexys-A7 development board. The manycore project, including HDL, scripts, and software, is available online at https://github.com/andersondomingues/orca-rt-tools. Simulation scripts require Questa and Vivado tools.

B. Case Study Application — Task Reallocation

The task reallocation subsystem of the ORCA manycore concerns moving tasks between PEs whenever tasks fail to meet deadlines. There is a threshold for each task, configured during application load to the manycore. In this case study, the subsystem will migrate the consumer task in a producer-consumer application. The manycore has a 2×3 dimension comprising one off-chip communication tile (#0) and five processing tiles (#1 to #5). A *producer* task in #1 sends packets at 3ms intervals to a *Consumer* task in #2, both registered within a *broker* in #3 under the same publish-subscribe topic. A *requester* in #5 commands a *spawner* in #2 to instantiate synthetic *dummy* tasks that execute dummy code (empty loops) every 100ms, progressively overloading #2. An *observer* in #2 monitors *consumer* deadline misses via the monitoring library; it notifies the *spawner* in #4 to spawn a replacement *consumer* in #4 when reaching 10% deadline misses. Figure 2 shows the step-by-step reallocation.

Figure 3 shows how the proposed framework models the task reallocation mechanism. The hardware layer is unused because the monitors collect data from software, and all actions regard the software layer as well. There is a single monitor, M_1, that tracks the number of missed deadlines in the *consumer* application. Evaluator E_1 uses the monitor information to trigger the decision logic D_1 whenever 10% of deadlines are missed over a 100ms observation period. Decision logic D_1 has a hard-coded plan to activate procedure P_1, which will execute actions A_1, A_2, and A_3 *in tandem*. Action A_1 notifies the task spawner to create a new *consumer* in #4, action A_2 kills the *consumer* in #2 (overloaded), and A_3 subscribes the new *consumer* to the data topic, restoring the application behavior.

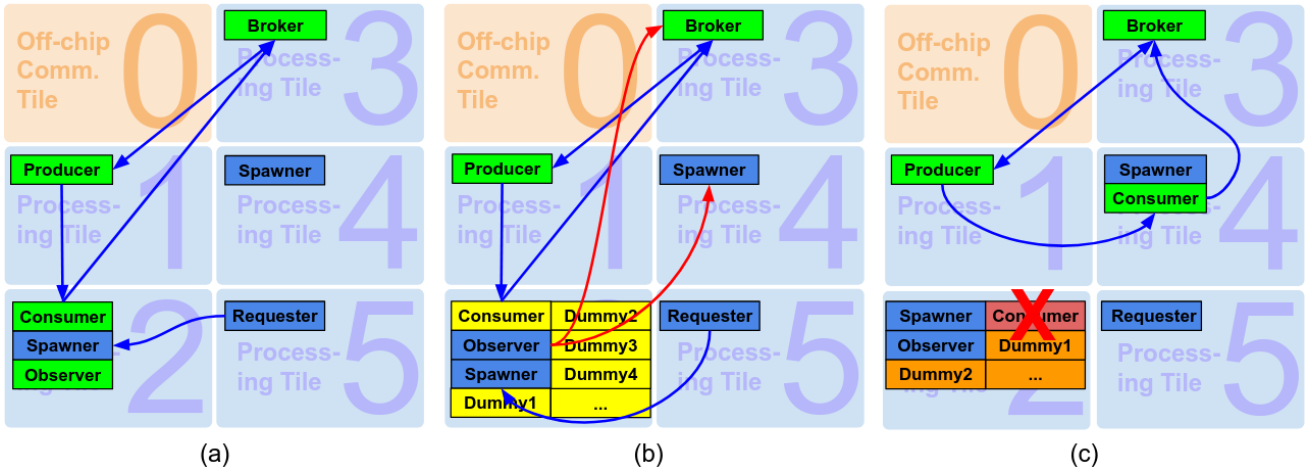

Fig. 2. The task reallocation mechanism performing step-by-step. First, a simple producer-consumer application is deployed to PEs #1 and #2 (**a**). Periodically, the requester in #5 activates the spawner in #2 (**b**). Eventually, #2 becomes overloaded and misses deadlines, triggering the task reallocation (**c**). Blue arrows indicate continuous communication, and red arrows indicate one-time communication. The color code for tasks is as follows: (green) healthy, (yellow) some deadlines missed, (orange) scheduling is unfeasible, (red) task removed from the task set (adapted from [11]).

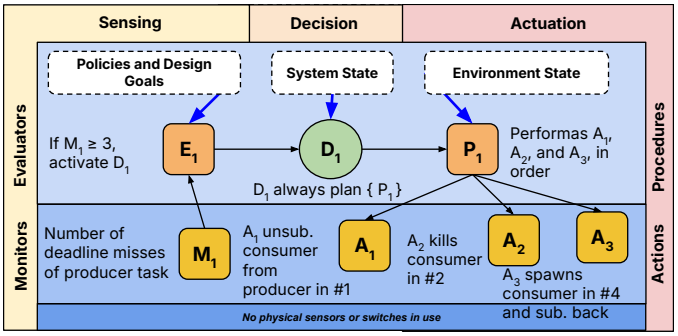

Fig. 3. The task reallocation mechanism modeled by the proposed framework. There is a single monitor M_1, which observes the number of deadline misses in a producer task. Evaluator E_1 waits for 10% of missing deadlines to activate decision logic D_1. There is only a single plan in this application, which will reallocate the task in another PE, represented by procedure P_1. Finally, actions A_1, A_2, and A_3 represent the steps for reallocating the task.

The *reaction time* of the task reallocation subsystem is conditioned on three events: (i) identification of the "need" for adaptation, (ii) definition of an adaptation plan, and (iii) execution of the plan. The "need" for adaptation corresponds to the time taken by E_1 to identify the missed deadline; failing to meet 10% of the period. The approximation $t(E_1) \equiv 3t(M_1)+k$ holds, where k is the execution time (code) of E_1, and t is the discrete time function. The *observer* task in #2 (E_1) operates on a fixed schedule, i.e. it is immune to CPU overload. As D_1 implements a static plan that always triggers P_1, $t(D_1) \equiv t(P_1)$. As P_1 sequentially executes A_1, A_2, and A_3, then $t(P_1) \equiv t(A_1)+t(A_2)+t(A_3)+j$, where j is the time taken by P_1 to orchestrate all actions. The system implement actions as follows: A_1 is a routine in the *observer* (#2), A_2 is implemented by the *spawner* task (#5), and A_3 is collateral to the startup of new consumer task (#4). The reaction time analysis for the task reallocation subsystem is shown in (1).

$$\begin{aligned} t &= t(E_1) + t(D_1) + t(P_1) \\ &= 3t(M_1) + k + t(A_1) + t(A_2) + t(A_3) + j \end{aligned} \tag{1}$$

VI. Conclusion

This paper surveyed the literature to extract the requirements for self-adaptiveness in SoCs, proposing a **framework for modeling self-adaptive systems**, exploring a multidimensional, layered approach. The framework comprises sensors, counters, monitors, and evaluators in the sensing dimension, and procedures, actions, and specialized hardware in the actuation dimension. At the decision dimension, decision logic connects sensing and actuation to form a closed-loop approach. The framework considers software, hardware, and logic layers, separating concerns for system components. Finally, policies, design goals, and system and environment states are inputs to the framework, also accounting for late changes in system specification.

The proposed framework is a promising solution for modeling self-adaptive systems. Due to space limitations, the paper presents a simple yet representative reallocation mechanism to demonstrate the framework's feasibility. Such a framework enables the study of the individual components of self-adaptive systems and has the potential to augment sensor-rich applications in the automotive, avionic, health, and military industries in the near future. Future work includes:

1) Improving the framework to include timing analysis to support real-time applications.
2) Developing case studies on more complex self-adaptive applications, prioritizing those with comprehensive sets of hardware sensors;
3) Incorporating knowledge from other domains [20,21] to improve modeling capabilities.
4) Exploring the traceability of project artifacts (e.g., source code, register-transfer level (RTL) design, and specifications) back to the building blocks of the framework, e.g., using annotations and meta-programming.

Acknowledgments

This work was financed in part by Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), 305621/2024-6; Fundação de Amparo à Pesquisa do Estado do Rio Grande do Sul (FAPERGS), 23/2551-0002200-1.

References

[1] M. Raeiszadeh, A. Ebrahimzadeh, R. H. Glitho, J. Eker, and R. A. F. Mini, "Real-Time Adaptive Anomaly Detection in Industrial IoT Environments," *CoRR*, vol. abs/2601.03085, 2026, https://doi.org/10.48550/arXiv.2601.03085.

[2] S. Macenski, T. Foote, B. P. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Sci. Robotics*, vol. 7, no. 66, 2022, https://doi.org/10.1126/scirobotics.abm6074.

[3] L. Prenzel, S. Hofmann, and S. Steinhorst, "Real-time Dynamic Reconfiguration for IEC 61499," in *ICPS*, 2022, pp. 1–6, https://doi.org/10.1109/ICPS51978.2022.9816872.

[4] F. Golpayegani, N. Chen, N. Afraz, E. Gyamfi, A. Malekjafarian, D. Schäfer, and C. Krupitzer, "Adaptation in Edge Computing: A Review on Design Principles and Research Challenges," *ACM Trans. Auton. Adapt. Syst.*, vol. 19, no. 3, pp. 19:1–19:43, 2024, https://doi.org/10.1145/3664200.

[5] G. M. Lipsa, A. Herkersdorf, W. Rosenstiel, O. Bringmann, and W. Stechele, "Towards a Framework and a Design Methodology for Autonomic SoC," in *ICAC*, 2005, pp. 391–392, https://doi.org/10.1109/ICAC.2005.61.

[6] U. Brinkschulte, J. Becker, and T. Ungerer, "CARUSO - an approach towards a network of low power autonomic systems on chips for embedded real-time applications," in *IPDPS*, 2004, pp. 124–129, https://doi.org/10.1109/IPDPS.2004.1303087.

[7] L. Guang, J. Plosila, J. Isoaho, and H. Tenhunen, *HAMSoC: A Monitoring-Centric Design Approach for Adaptive Parallel Computing*. CRC Press, 2012, ch. 6, pp. 136–164, https://doi.org/10.1201/9781315217826.

[8] S. Sarma, N. Dutt, P. Gupta, N. Venkatasubramanian, and A. Nicolau, "Cyberphysical-system-on-chip (CPSoC): a self-aware MPSoC paradigm with cross-layer virtual sensing and actuation," in *DATE*, 2015, https://dl.acm.org/doi/10.5555/2755753.2755895.

[9] A. v. Renteln, U. Brinkschulte, D. Kramer, W. Karl, C. Schuck, and J. Becker, "Digital on-demand computing organism - interaction between monitoring and middleware," in *ISORC*, 2011, p. 189–196, https://doi.org/10.1109/ISORC.2011.31.

[10] M. Ruaro, L. L. Caimi, V. Fochi, and F. G. Moraes, "Memphis: a framework for heterogeneous many-core SoCs generation and validation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 23, no. 4, pp. 103–122, 2019, https://doi.org/10.1007/s10617-019-09223-4.

[11] A. R. P. Domingues, J. Benno, A. M. Amory, and F. G. Moraes, "ORCA RT-Bench: A Reference Architecture for Real-Time Scheduling Simulators," in *SBESC*, 2021, pp. 1–6, https://doi.org/10.1109/SBESC53686.2021.9628369.

[12] A. Jantsch, N. D. Dutt, and A. M. Rahmani, "Self-Awareness in Systems on Chip - A Survey," *IEEE Des. Test*, vol. 34, no. 6, pp. 8–26, 2017, https://doi.org/10.1109/MDAT.2017.2757143.

[13] J. Kephart and D. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003, https://doi.org/10.1109/MC.2003.1160055.

[14] A. Bouajila, A. Bernauer, A. Herkersdorf, W. Rosenstiel, O. Bringmann, and W. Stechele, "Error Detection Techniques Applicable in an Architecture Framework and Design Methodology for Autonomic SoCs," in *Biologically Inspired Cooperative Computing*. Boston, MA: Springer US, 2006, pp. 107–113, https://doi.org/10.1007/978-0-387-34733-2_11.

[15] J. Zeppenfeld, A. Bouajila, W. Stechele, and A. Herkersdorf, "Learning Classifier Tables for Autonomic Systems on Chip," in *Proceedings of the ARCS Workshop on Organic Computing*, 2010, pp. 771–778, https://dl.gi.de/handle/20.500.12116/21283.

[16] J. Zeppenfeld and A. Herkersdorf, "Applying autonomic principles for workload management in multi-core systems on chip," in *ICAC*, 2011, p. 3–10, https://doi.org/10.1145/1998582.1998586.

[17] C. Schuck, S. Lamparth, and J. Becker, "artNoC - A Novel Multi-Functional Router Architecture for Organic Computing," in *FPL*, 2007, pp. 371–376, https://doi.org/10.1109/FPL.2007.4380674.

[18] S. Johann, "HF-RISC SoC," 2024, online. Visited in January 2024. https://github.com/sjohann81/hf-risc.

[19] J. Arenas, C.-H. Huang, K. Patino-Sosa, J.-J. Park, H. S. Ozturk, and V. Sathe, "A Dynamically Reconfigurable Digital-Integrated Voltage-Regulator Fabric for Energy-Efficient DVFS in Multi-Domain soCs," in *ISSCC*, 2025, pp. 1–3, https://doi.org/10.1109/ISSCC49661.2025.10904508.

[20] R. Iida, K. Oishi, and H. Nakagawa, "An Efficient Runtime Verification Toolkit for Self-Adaptive Systems Addressing Runtime System Model Changes," *IEEE Access*, pp. 1–1, 2026, https://doi.org/10.1109/ACCESS.2026.3682704.

[21] K. Furukawa, H. Nakagawa, and T. Tsuchiya, "A Self-Adaptive Framework for Deploying Machine Learning Systems Without Ground-Truth Data at Runtime," *IEEE Access*, vol. 14, pp. 30 309–30 326, 2026, https://doi.org/10.1109/ACCESS.2026.3663383.